RetailLens

E-commerce Data Warehouse & Price Intelligence Pipeline

Web-to-Warehouse Product Intelligence Pipeline

Project Documentation - Parts 1 to 3 (Revised Edition)

1. Project Planning & Management | 2. Literature Review | 3. Requirements Gathering

Digital Egypt Pioneers Initiative - Phase 5 | Graduation Project
Delivered by: CLS Learning Solutions

Team: [Team Name] | GitHub: [Repository URL] | Date: October 2026

Part 1: Project Planning & Management

1.1 Project Idea and Problem Statement

Online retailers need to continuously track product information (names, categories, prices, ratings, availability) from several places: their own internal catalog and permitted public web sources such as competitor sites or public APIs. Today this work is manual, slow and inconsistent: the same product appears under different names, prices are written in different formats and currencies, duplicates are common, and nobody keeps the price history, so it is impossible to answer questions such as “When did this price change?” or “Which products disappeared this month?”

RetailLens solves this by building an automated, end-to-end data pipeline. It extracts product data from permitted sources, cleans and standardizes it, detects duplicates, compares it with the internal product catalog, loads it into an analytical database (a data warehouse), stores a historical snapshot of prices on every run, and exposes SQL analysis and dashboards. The whole process is orchestrated with Apache Airflow so it runs on a schedule with no manual work.

1.2 Project Objectives

- **O1 - Ingestion:** Collect product name, category, price, rating, availability and product URL from at least two permitted sources (one scraped website and one public API), respecting robots.txt, rate limits and terms of use.
- **O2 - Standardization:** Clean product names and categories, and normalize prices and currencies into one reporting currency.
- **O3 - Deduplication:** Detect and merge duplicate products within and across sources.
- **O4 - Integration:** Compare scraped records with the internal product catalog (matching, new, removed, changed).
- **O5 - Warehousing:** Load clean data into PostgreSQL using a star schema, with a historical price-snapshot table.
- **O6 - Analytics:** Use SQL to identify price changes, new products, removed products and category changes, and present them in a dashboard.
- **O7 - Automation and Quality:** Orchestrate the pipeline with Airflow and enforce automated data-quality checks on every run.
- **O8 (optional) - Serving:** Expose the latest product data through a REST API.

1.3 Project Scope

In Scope

- Scraping/API ingestion from permitted sources only.
- Cleaning, normalization and deduplication.
- Internal catalog comparison (simulated catalog table).
- PostgreSQL data warehouse and historical price table.
- SQL analysis, data-quality checks and an Airflow DAG.
- Dashboard/visualization and full documentation.
- Optional REST API (FastAPI).

1.4 Proposed Data Sources (to be confirmed by the team)

Source	Type	Why it is suitable
books.toscrape.com	Web scraping (BeautifulSoup / Scrapy)	A sandbox website built specifically for scraping practice. Provides name, category, price (GBP), star rating, availability and URL for approximately 1,000 products.
DummyJSON Products API (or Fake Store API)	Public REST API (JSON)	Free API with products, categories, prices (USD), ratings and stock. Shows API ingestion and currency normalization.
Internal catalog (simulated)	CSV loaded into PostgreSQL	Represents the retailer's existing product database, used for the comparison step.

Note on price history: Practice sites are mostly static, so real price changes will not appear. To demonstrate historical tracking, the team will include a documented price simulator that applies small random changes, additions and removals between runs. This will be clearly stated as simulated data in the final report.

1.5 Technology Stack

Layer	Tool	Purpose
Language	Python 3.11	Scraping, transformation, DQ checks
Extraction	BeautifulSoup, Scrapy, requests	HTML scraping and API calls
Transformation	pandas, RapidFuzz	Cleaning, normalization, fuzzy duplicate detection
Database	PostgreSQL	Data warehouse (staging + star schema)
Orchestration	Apache Airflow	Scheduling and monitoring the DAG
Data quality	Custom checks (or Great Expectations)	Validate every load
Visualization	Power BI / Metabase / Streamlit	Dashboard
Serving (optional)	FastAPI	REST API for latest product data
DevOps	Docker Compose, GitHub	Reproducible environment and version control

1.6 System Architecture (Expanded)

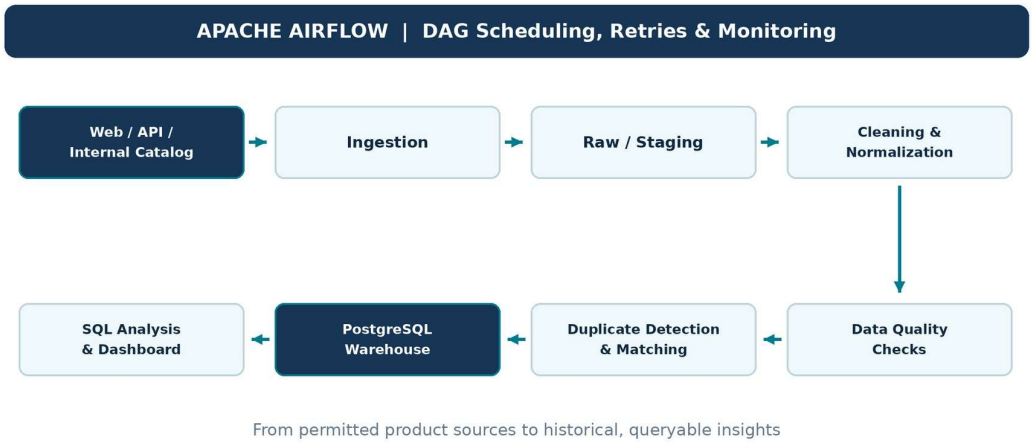
RetailLens uses a layered architecture. Each layer receives data from the preceding layer, performs a defined task, and passes validated outputs to the next layer. Apache Airflow orchestrates these layers; it is not itself a data store.

1.6.1 Core Components

1. **Data Sources:** A permitted scraped website, a public product API and a simulated internal catalog (CSV).
2. **Ingestion Module:** Python scraper and API client to retrieve product records; manages source configuration, rate limits and connection errors.
3. **Raw / Staging Layer:** Temporarily stores raw HTML, JSON or CSV-derived records before they are standardized.
4. **Processing Module:** Cleans product names, standardizes categories and numeric prices, and handles missing or invalid fields.

5. **Data Quality Module:** Checks completeness, uniqueness, valid ranges, source references and consistency before the final warehouse load.
6. **Product Matching Module:** Detects duplicates and matches external records to the existing internal product catalog.
7. **PostgreSQL Warehouse:** Stores cleaned dimensions and facts, including historical product-price snapshots.
8. **Analytics & Visualization:** SQL queries/views and a dashboard surface price changes, new or removed products, and category changes.
9. **Apache Airflow:** Schedules, orders, retries and monitors pipeline tasks from ingestion to reporting.

1.6.2 Architecture Diagram



RetailLens system architecture

1.6.3 Layer Responsibilities

Layer	Content	Output
1. Sources	Website (scraper), public API, internal catalog	Raw HTML / JSON / CSV
2. Ingestion	Python scraper and API client with rate limiting and robots.txt checks	Raw files and staging records (stg_*)
3. Transformation	Clean names/categories; normalize prices/currencies; deduplicate; match with internal catalog	Clean, standardized records
4. Quality gate	Row counts, nulls, valid ranges, uniqueness and freshness	Pass/fail results and rejected-record details
5. Warehouse	Load dimensions and facts (star schema), save price snapshots	dim_product, dim_category, dim_source, fact_price_snapshot
6. Analytics & Serving	SQL views, dashboard and optional REST API	Insights for the business
Orchestration	Airflow DAG governing dependencies and execution status	Scheduled or manual runs with logs

1.6.4 End-to-End Data Flow

Websites / APIs / Internal Catalog → Ingestion → Raw / Staging Data → Cleaning & Normalization → Data Quality Checks → Duplicate Detection & Product Matching → PostgreSQL Warehouse → Price History & Change Detection → SQL Analysis → Dashboard.

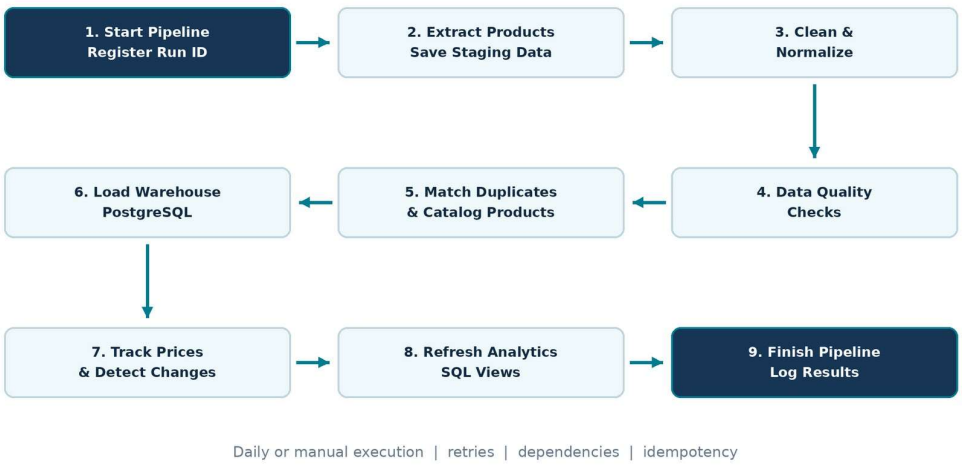
1.7 Apache Airflow Orchestration (Detailed)

This section incorporates the operational DAG requirements from the Arabic SRS. Airflow coordinates the entire ETL process and provides task-level status, retries and logs.

1.7.1 DAG Tasks

Order	Airflow Task	Responsibility
1	start_pipeline	Start the workflow and register a unique run ID.
2	extract_products	Fetch product records from permitted websites/APIs and preserve raw/staging data.
3	clean_normalize	Clean names, standardize categories, normalize prices and currencies.
4	data_quality_checks	Validate records and isolate invalid data before the final load.
5	match_duplicates	Detect repeated products and match source records to the internal catalog.
6	load_warehouse	Load valid products, categories, sources and related warehouse records into PostgreSQL.
7	track_prices_and_changes	Save historical price snapshots and identify new, removed or changed records.
8	refresh_analytics	Refresh the analytical SQL outputs used by the dashboard.
9	finish_pipeline	Record run status, row counts, execution results and errors.

1.7.2 DAG Execution Flow



RetailLens Airflow DAG flow

1.7.3 Scheduling and Operational Rules

- **Schedule:** One automatic daily run is proposed; the DAG can also be triggered manually.
- **Retries:** Temporarily failed tasks can retry according to configured retry limits.
- **Dependencies:** A dependent task must not proceed after an upstream critical failure.

- **Logging:** Each step records execution status, errors and useful processing counts.
- **Idempotency:** Re-running the same logical run must not create duplicated product rows, price snapshots or change events.
- **Data safety:** Failed or incomplete extraction must not be interpreted as evidence that products were removed.
- **History:** Price snapshots preserve previously observed prices rather than overwriting all historical values.

1.7.4 Expected Airflow Deliverable

A runnable product_pipeline_dag.py file connecting the ingestion, processing, quality, matching, loading and analytics functions. The Airflow interface must show task status and logs for a successful end-to-end run.

1.8 Proposed Project Structure

The following repository layout is adapted from the Arabic SRS. It keeps responsibilities separate so new sources and transformations can be added without rewriting the whole pipeline.

```
retail-lens/
|
|-- dags/
|   `-- product_pipeline_dag.py
|
|-- src/
|   |-- ingestion/
|   |   |-- api_client.py
|   |   `-- scraper.py
|   |
|   |-- processing/
|   |   |-- cleaning.py
|   |   |-- normalization.py
|   |   `-- matching.py
|   |
|   |-- validation/
|   |   `-- quality_checks.py
|   |
|   |-- database/
|   |   |-- connection.py
|   |   `-- loader.py
|   |
|   `-- analytics/
|       `-- change_detection.py
|
|-- sql/
|   |-- schema.sql
|   `-- analysis.sql
|
|-- dashboards/
|-- tests/
|-- docs/
|   |-- SRS.md
|   |-- architecture.md
|   `-- setup.md
|
|-- docker-compose.yml
|-- requirements.txt
|-- .env.example
|-- .gitignore
`-- README.md
```

Folder responsibilities: dags/ defines workflow execution; src/ingestion/ gathers data; src/processing/ cleans, normalizes and matches records; src/validation/ runs quality checks; src/database/ handles PostgreSQL access and loading; src/analytics/ identifies changes; sql/ stores schema and analytical queries; dashboards/ contains visualization work; tests/ holds tests; docs/ contains project documentation.

The structure is a recommended starting point and may be adjusted during implementation.

1.9 Risk Management

Risk	Likelihood	Impact	Mitigation
Website blocks scraper or changes HTML structure	Medium	High	Use sandbox/permitted sources, polite delays, modular selectors, save raw copies, add API fallback.
Legal/ethical issues with scraping	Low	High	Check robots.txt and terms before scraping; use only permitted sources; no personal data.
Dirty or inconsistent data	High	Medium	Cleaning rules, data-quality checks, rejected-records table for review.
Team members are new to Airflow/SQL/Docker	High	Medium	Early learning sessions, simple DAG first, pair programming, ask the Technical Instructor.
No real price changes appear in static sources	High	Medium	Documented price simulator to generate realistic history.
Integration problems between modules	Medium	Medium	Agree on data contracts (column names/types) early; use shared Docker environment.
Time shortage before final deadline	Medium	High	Prioritize must-have features; REST API is optional; weekly progress review.
Merge conflicts / lost work	Medium	Low	Single GitHub repo, feature branches, pull requests, regular commits.

Part 2: Literature Review

This section reviews existing tools and approaches related to RetailLens: commercial and free price-monitoring products, the engineering practices used to build such pipelines, and what the team learns from each. Feature descriptions are based on each product's public website and should be re-verified by the team before final submission.

2.1 Similar Solutions

Solution	What it does	Strengths	Limitations	Lesson for RetailLens
Keepa	Tracks Amazon product price history	Long price history, easy-to-read charts, alerts.	Limited to Amazon; closed system; no access to the retailer's own catalog or warehouse.	Historical price charts are valuable; we store history in our own warehouse.
CamelCamelCamel	Amazon price tracker with price-drop notifications.	Simple, free for consumers.	Consumer tool, Amazon only, no analytics layer.	Alerting on price change is a good future extension.
Prisync	Competitor price tracking and dynamic pricing for e-commerce stores.	Business-oriented, supports many competitors, dashboards.	Paid subscription; a black box; limited control of data model.	Shows the business need; we offer an open, customizable pipeline.
Price2Spy	Price monitoring and repricing across competitor websites.	Wide site coverage, reports.	Paid service; not designed to join with the retailer's own warehouse for custom SQL.	Competitor comparison with internal data is the key value we replicate on a small scale.
Generic Scrapy/BeautifulSoup price trackers (open-source projects)	Scripts that scrape prices and store them in CSV/SQLite.	Free, simple, educational.	Usually no orchestration, no warehouse design, no data-quality checks, no duplicate handling.	We add proper ETL, star schema, Airflow and quality gates.
Managed ELT tools (Airbyte, Fivetran)	Move data from APIs/databases into warehouses automatically.	Many connectors, reliable, low code.	Not built for scraping arbitrary sites; cost; less learning value.	Confirms the Extract-Load-Transform pattern; we build custom code to learn the full flow.

2.2 Technical Approaches and Best Practices

- **ETL vs ELT:** Classic ETL cleans data before loading; modern ELT loads raw data first and transforms inside the warehouse. RetailLens uses a hybrid: raw data is kept in staging tables, then Python and SQL transform it into the warehouse.
- **Dimensional modeling (Kimball):** Star schemas with fact and dimension tables make analytical SQL simple and fast. We follow this model for products, categories, sources and price snapshots.
- **Slowly Changing Dimensions (SCD Type 2) and snapshots:** To keep history, changes are stored as new rows with validity dates instead of overwriting old values. We use a snapshot fact table for prices and SCD2 ideas for category changes.
- **Fuzzy matching for duplicates:** Exact comparison fails for product names with different spelling or formatting. String-similarity libraries (for example RapidFuzz) with a threshold, combined with normalized names, are standard practice.
- **Workflow orchestration:** Apache Airflow defines pipelines as DAGs with retries, scheduling and monitoring, which is the industry standard for batch pipelines.

- **Ethical scraping:** Follow robots.txt (standardized as RFC 9309), use delays between requests, identify the scraper with a User-Agent, prefer official APIs when available, and respect website terms.
- **Data quality:** Automated checks (completeness, uniqueness, valid ranges, freshness) placed between transformation and loading prevent bad data from reaching analytics.

2.3 Comparison with RetailLens

Feature	Keepa / CCC	Prisync / Price2Spy	Simple open-source scripts	RetailLens
Multiple data sources (web + API + internal)	No	Partial	Rare	Yes
Join with internal product catalog	No	Limited	No	Yes
Own data warehouse with star schema	No	No	No	Yes
Historical price table	Yes	Yes	Sometimes	Yes
Automated duplicate detection	N/A	Hidden	No	Yes
Data-quality checks	N/A	Hidden	No	Yes
Orchestration with Airflow	N/A	N/A	No	Yes
Custom SQL analysis	No	Limited reports	Manual	Yes
Cost	Free/paid	Paid	Free	Free (open-source)

Summary of differences: Commercial tools solve the monitoring problem but are closed, paid and not connected to a retailer's own warehouse. Simple scripts are free but lack structure and reliability. RetailLens sits in between: a small, open, well-engineered pipeline that gives the retailer full control of its data and a clear learning value for the team.

Part 3: Requirements Gathering

3.1 Stakeholders and Their Needs

Stakeholder	Role	Main needs
Retail business / Category managers	Primary users of insights	Know when competitor or source prices change, which products are new or removed, and how categories shift.
Pricing and marketing analysts	Run SQL and dashboards	Clean, consistent, historical data with a clear schema; easy-to-use dashboard.
Data engineers (maintainers)	Operate the pipeline	Modular code, logs, retries, monitoring, easy configuration.
Internal catalog / IT team	Own the existing product database	Safe comparison with their data without modifying it.
Management	Decision makers	Reliable, up-to-date KPIs and reports.
Developers / external systems (optional)	Consume the REST API	Stable endpoints returning the latest product data in JSON.
Project supervisors (CLS / Technical Instructor)	Evaluators	Complete deliverables, documentation and a working demo.

3.2 Functional Requirements

ID	Requirement	Priority
FR-01	The system shall extract product name, category, price, rating, availability and product URL from the scraped website.	Must
FR-02	The system shall extract the same fields from a public API source.	Must
FR-03	The scraper shall check robots.txt, apply delays between requests and use a clear User-Agent.	Must
FR-04	The system shall clean product names (trim, case, special characters, extra spaces) and standardize category names.	Must
FR-05	The system shall normalize prices to numeric values and convert currencies to one reporting currency using configurable rates.	Must
FR-06	The system shall detect duplicate products within and across sources using exact and fuzzy matching.	Must
FR-07	The system shall compare scraped records with the internal product catalog and flag matched, new, removed and changed products.	Must
FR-08	The system shall load data into PostgreSQL staging tables and then into the warehouse (star schema).	Must
FR-09	The system shall store a historical price snapshot with a timestamp on every pipeline run.	Must
FR-10	The system shall provide SQL queries/views to identify price changes, new products, removed products and category changes.	Must
FR-11	The system shall run data-quality checks on every load and log the results; critical failures shall stop the load.	Must
FR-12	The pipeline shall be orchestrated with an Airflow DAG with scheduling, retries and task dependencies.	Must
FR-13	The system shall provide a dashboard with price trends, new/removed products, category changes and price comparison.	Must
FR-14	The system shall keep rejected/invalid records in a separate table for review.	Should
FR-15	The system shall expose the latest product data through a REST API (for example GET /products, GET /products/{id}, GET /products/{id}/price-history).	Could (optional)

ID	Requirement	Priority
FR-16	The system shall include a documented price simulator to generate realistic changes for demonstration.	Should

3.3 Non-Functional Requirements

Category	Requirement
Performance	A full run over about 1,000 products shall finish in under 15 minutes; common analytical queries shall return in under 5 seconds (indexes on keys and dates).
Reliability	Failed tasks shall retry automatically (for example 3 times); the pipeline shall be idempotent so re-running a day does not create duplicates.
Data quality	At least 98% completeness of required fields; no negative or zero prices; unique product keys per source; data freshness checked each run.
Security	Credentials stored in environment variables or Airflow connections, never in GitHub; least-privilege database users; no personal data collected.
Usability	Dashboard understandable by non-technical users; clear README with setup steps; sensible naming of tables and columns.
Scalability	Modular design so new sources can be added by writing a new extractor only; schema supports more products and longer history.
Maintainability	Clean, commented Python code; configuration in one place; logging; version control with pull requests.
Portability	The entire environment shall start with Docker Compose on any machine.
Legal and ethical compliance	Only permitted sources; respect robots.txt, rate limits and terms of use; attribute data sources in documentation.
Observability	Run logs, task durations and data-quality results stored for review.

3.4 Data Requirements

Core fields collected for every product and the planned warehouse tables:

Field	Type	Notes
product_name	Text	Cleaned and standardized; a normalized version is kept for matching.
category	Text	Standardized category name.
price	Numeric	Original price and currency kept; converted price in reporting currency.
rating	Numeric (0-5)	Nullable if not provided.
availability	Text/Boolean	For example in stock / out of stock, plus quantity if available.
product_url	Text	Used as a natural key for each source.
source_id, scraped_at	Key, Timestamp	Track where and when each record was collected.

Table	Purpose
stg_products_raw / stg_products_clean	Staging layer for raw and cleaned records.
dim_product	One row per unique product (surrogate key, natural key, cleaned name, URL).
dim_category	Category dimension with history of category assignments.
dim_source	Data sources (website, API, internal catalog).
dim_date	Calendar dimension for time analysis.
fact_price_snapshot	Historical price, rating and availability for each product on each run.
dq_check_results, rejected_records	Data-quality log and invalid records.

3.5 User Stories

- As a pricing analyst, I want to see all products whose price changed since the last run, so I can react to market changes.
- As a category manager, I want to see new and removed products per category, so I can update our assortment.
- As a data engineer, I want failed runs to retry and send logs, so I can fix problems quickly.
- As a manager, I want a dashboard with price trends over time, so I can make decisions without writing SQL.
- As a developer, I want a REST API returning the latest product data, so other systems can use it (optional).

3.6 Assumptions and Constraints

- Only publicly permitted sources will be used; if a source forbids scraping, it will not be used.
- The internal catalog is simulated with a CSV/PostgreSQL table because no real retailer data is available.
- Price changes are partly simulated, and this will be clearly disclosed in the final report.
- The project must be completed within the graduation-project timeline using free and open-source tools.
- The team is at the beginning of its data career, so the design favors simplicity and clarity over complexity.

3.7 Acceptance Criteria

- Running the Airflow DAG once loads data from all sources into the warehouse without manual steps.
- A second run adds a new price snapshot and does not duplicate products.
- The four SQL analyses (price changes, new products, removed products, category changes) return correct results on test data.
- Data-quality checks run on each load, and a deliberately broken input is detected and reported.
- The dashboard displays the main insights, and the complete documentation is available in the GitHub repository.